MetroGuard AI

Comprehensive Technical Architecture & Implementation Report — Phase-I Crowd Density Prototype

Focus: Real-Time Occupancy & Crowd Flow Monitoring **Detection:** YOLOv11s (COCO Class 0: Person)

Tracking: ByteTrack Association **UI:** Streamlit Operator Console **Target Domain:** Transit & Metro Security

1. Executive Summary & Project Purpose

Modern urban rail and rapid transit networks transport millions of commuters daily. Overcrowded platforms, stairwells, and turnstile concourses represent severe operational vulnerabilities that can rapidly escalate into stampedes, accidental track falls, boarding delays, or passenger distress. While high-definition CCTV systems are ubiquitous across transit stations, human security operators cannot concurrently monitor dozens of camera feeds with sustained vigilance.

MetroGuard AI (Phase-I) delivers an automated, real-time computer vision subsystem engineered to measure, classify, and track commuter crowd levels within operator-designated *Regions of Interest (ROI)*. The system continuously ingests surveillance video streams, detects and tracks human commuters across sequential frames, filters individuals based on ground-plane physical contact, and produces dynamic alert telemetry (LOW, MEDIUM, HIGH) via an interactive operations console and an automated command-line processing pipeline.

> **Engineering Discipline: Explicit Phase-I Scope & Scientific Honesty**
>
> Unlike conventional speculative prototypes that claim automated violence recognition, anomaly detection, or abandoned baggage identification without validated ground truth, **Phase-I is deliberately and rigorously restricted to verified person counting and occupancy classification**. Furthermore, the system measures *image-space occupancy* (people per ROI-megapixel) rather than physical density (people/m^2), acknowledging that true physical density requires extrinsic camera calibration and homography matrices.

2. Technologies, Frameworks & Deep Learning Models

Every software library, neural architecture, and utility in MetroGuard AI was selected to balance detection recall, tracking consistency, frame processing latency, and operational reproducibility.

Technology / Tool	Role in MetroGuard AI	Technical Significance & Rationale	Implementation Detail
YOLOv11s (Ultralytics 8.4)	Primary deep neural detector for extracting bounding boxes of commuters.	State-of-the-art one-stage detector combining C3k2 building blocks, SPPF (Spatial Pyramid Pooling - Fast), and an anchor-free detection head. Delivers higher mAP at faster inference speeds compared to YOLOv8 and YOLOv5.	Configured via `ultralytics.YOLO()` with weights resolution in `locate_default_model()`. Restricted to `classes=[0]` (COCO `person`) to prevent false positive allocations.
ByteTrack Framework	Multi-Object Tracking (MOT) across consecutive frames.	Solves the occlusion problem in crowded environments. Unlike trackers that discard low-confidence detections, ByteTrack associates both high-confidence and low-confidence boxes using Kalman filters and Hungarian matching, maintaining stable person IDs through overlaps.	Executed natively through `model.track(..., persist=True, tracker="bytetrack.yaml")`. Track IDs are stored and assigned to per-frame telemetry.
OpenCV (cv2 5.0)	Video decoding, frame extraction, spatial graphics rendering, and video encoding.	High-speed C++ computer vision engine. Efficiently decodes H.264/MP4 streams, applies transparent ROI alpha overlays, draws color-coded bounding boxes and HUD banners, and writes annotated videos.	`cv2.VideoCapture` for frame iteration; `cv2.rectangle` and `cv2.addWeighted` for semi-transparent zone shading; `cv2.VideoWriter` for annotated MP4 creation.
Streamlit 1.65	Interactive web console for transit operators and supervisors.	Enables rapid reactive UI development in pure Python with zero front-end overhead. Provides dynamic real-time frame streaming, interactive sliders for live ROI adjustment, metric displays, and tabular logs.	`dashboard.py` utilizes `st.empty()` containers for atomic DOM element updates without entire page flickering; multi-column layout for side-by-side feed and telemetry.
NumPy 2.5 & Pandas 3.0	Vectorized geometric computations and time-series event telemetry.	NumPy accelerates bounding box matrix calculations (footpoint slicing, coordinates transformations). Pandas organizes state-transition timestamps and powers the empirical evaluation suite.	`boxes.xyxy.cpu().numpy()` for vector manipulation; `pd.DataFrame` for event logs and accuracy evaluation metric exports.
PyTorch 2.14	Deep learning tensor runtime.	Executes the YOLOv11 forward-pass, automatically utilizing NVIDIA CUDA GPU acceleration when present and seamlessly falling back to optimized CPU vector instructions.	Underlying execution engine for `ultralytics`.
LAP (Lapjv)	Linear Assignment Problem solver.	Computes optimal bipartite matching between predicted Kalman filter trajectories and detected bounding boxes in $O(N^3)$ polynomial time.	Core backend algorithmic engine utilized by ByteTrack for identity association.

3. Mathematical & Algorithmic Foundations

3.1 Normalized Coordinate Space and Region of Interest (ROI)

Surveillance video resolutions vary across transit deployments (e.g., 720p, 1080p, 4K). To maintain camera-agnostic portability, the Region of Interest is defined in normalized unit space:

$$\text{ROI} = \{ (x, y) \in [0, 1] \times [0, 1] \mid \text{left} \le x \le \text{right} \ \text{ and } \ \text{top} \le y \le \text{bottom} \}$$

When applied to a video frame of pixel dimensions W × H, the absolute pixel boundaries are computed via rounding:

$$(x_min, y_min, x_max, y_max) = (\text{round}(\text{left} \cdot W), \text{round}(\text{top} \cdot H), \text{round}(\text{right} \cdot W), \text{round}(\text{bottom} \cdot H))$$

3.2 Footpoint Formulation: Ground-Contact Geometric Filtering

A frequent source of counting error in overhead surveillance systems is upper-body perspective overlap: an individual walking *outside* the monitored zone may cast a bounding box that intersects the ROI if their head or torso leans across the boundary.

> **The Footpoint Advantage Over Bounding Box Centroids**
>
> Standard naive counting checks whether the box centroid ((x1+x2)/2, (y1+y2)/2) is inside the polygon. In CCTV cameras angled obliquely downwards, a person standing outside the ROI will have their torso falsely projected inside. MetroGuard AI isolates the **bottom-center point (footpoint)** as a reliable proxy for physical floor contact:
>
> $$p_foot = ((x1 + x2) / 2, \ \ y2)$$
>
> The containment predicate evaluates strictly:
>
> $$\text{Contains}(\text{ROI}, \text{box}) \Leftrightarrow (x_min \le (x1 + x2)/2 \le x_max) \wedge (y_min \le y2 \le y_max)$$

3.3 Operational Density State Machine

Raw person counts are mapped into actionable operational security states via configurable boundary thresholds (T_med and T_high):

$$\text{State}(C) = \text{LOW} \ \text{ if } \ C < T_med \quad | \quad \text{MEDIUM} \ \text{ if } \ T_med \le C < T_high \quad | \quad \text{HIGH} \ \text{ if } \ C \ge T_high$$

Whenever State(C_t) ≠ State(C_t-1), an asynchronous event trigger is dispatched with timestamp telemetry to alert the control room of an emerging bottleneck.

3.4 Image-Space Occupancy Metric

Because camera calibration matrices and ground-plane homography are not assumed in Phase-I, the project reports normalized image-space occupancy:

$$\text{Occupancy per Megapixel} = \text{Count} / \text{Area_ROI(Mpx)} = \text{Count} / [(x_max - x_min) \cdot (y_max - y_min) / 1{,}000{,}000]$$

4. Detailed Codebase & File-by-File Analysis

The codebase is architected with clear separation between core computer vision algorithms, interactive graphical interfaces, headless execution utilities, and statistical evaluation suites.

4.1 `metroguard/monitor.py` (The Core Engine)

This module implements all detection, tracking, geometric filtering, and annotation primitives.

- **`ROI` (frozen dataclass):** Stores normalized coordinates `left, top, right, bottom` with boundary validation (asserting 0 ≤ left < right ≤ 1). Provides `pixels(width, height)` for absolute coordinate conversion, `contains_footpoint(box, width, height)` for ground-contact testing, and `area_megapixels()`.
- **`MonitorConfig` (frozen dataclass):** Encapsulates runtime parameters: `model_path`, `confidence` (default 0.35), `imgsz` (default 1280 to retain high recall for distant passengers), `medium_min` (default 8), `high_min` (default 16), `roi`, and `tracker` (`bytetrack.yaml`).
- **`FrameResult` (dataclass):** Output payload per processed frame, containing `frame_index`, `timestamp_sec`, the annotated `frame` (NumPy ndarray), `person_count`, `density_level`, `occupancy_per_mpx`, list of `tracked_ids`, and state transition `event` message.
- **`locate_default_model()`:** Robust weight resolution utility. Sequentially searches the `METROGUARD_MODEL` environment variable, local project `models/yolo11s.pt`, and the local legacy cache, falling back to automatic download.
- **`CrowdMonitor` (Core Orchestration Class):**
  - `__init__(config)`: Instantiates YOLO and isolates the configuration directory to project-local `.ultralytics` to prevent write permission failures in enterprise/lab environments.
  - `analyze(frame, index, timestamp)`: Calls `model.track(classes=[0], conf=conf, imgsz=imgsz)`, parses bounding boxes and track IDs, executes footpoint filtering inside the ROI, computes occupancy, updates the state transition machine, and renders annotations.
  - `stream(source, max_frames)`: Generator yielding `FrameResult` objects frame-by-frame from video files or live RTSP/webcam feeds.
  - `_annotate(frame, selected, count, level)`: Renders a color-coded transparent HUD overlay (Green for LOW, Orange for MEDIUM, Crimson for HIGH), active ROI boundaries, bounding boxes with track IDs and confidence, and an executive top status banner.

4.2 `dashboard.py` (Streamlit Operator Console)

Provides a browser-based operations interface designed for transit monitoring rooms.

- **Layout Structure:** Responsive wide-screen layout. Features a dedicated *Detection & Zone Settings* sidebar for tuning thresholds, confidence, inference resolution, and ROI boundaries.
- **Centered Ingestion Portal:** The main center area provides side-by-side video upload (drag-and-drop) and local video path entry (with auto-complete placeholders for sample crowd clips), topped by a prominent "Start Monitoring" control.
- **Real-Time Display Loop:** Uses Streamlit's `st.empty()` reactive placeholders to update the video stream and metric cards (*People in ROI*, *Density Status*, *System Heartbeat*) frame-by-frame without browser page reloads.
- **Live Transition Event Log:** Dynamically populates a descending chronological event table highlighting every density state transition with sub-second timestamps.

4.3 `run_video.py` (Headless Batch Processor)

Engineered for backend processing, server scripts, and CI/CD pipelines.

- Accepts input sources (local file, RTSP network camera, or device index) and parameters via `argparse`.
- Streams frames through `CrowdMonitor`, writing the annotated visual stream to disk via `cv2.VideoWriter` using the `mp4v` codec.
- Logs real-time density change events to stdout and emits summary statistics upon completion.

4.4 `evaluate_counting.py` (Empirical Scoring Suite)

Provides a scientifically defensible method for measuring counting accuracy without requiring external ground-truth bounding-box annotations.

- `prepare` **sub-command:** Uniformly samples N frames across the video. Exports raw frames, model-annotated frames, and `roi_guides/` (which show *only* the ROI boundary with no model predictions to eliminate human confirmation bias during manual counting). Generates a blank `count_labels.csv`.
- `score` **sub-command:** Compares human counts against model predictions, calculating MAE, RMSE, Mean Signed Error (detecting under-counting or over-counting bias), WAPE, and overall accuracy percentage.

5. Empirical Evaluation Methodology & Metrics

Object detection benchmark metrics like mAP (Mean Average Precision) require fully annotated bounding boxes for every person in every frame. In real-world metro surveillance deployments, obtaining box annotations is prohibitively expensive. MetroGuard AI Phase-I implements an **ROI Counting Accuracy Protocol**:

Metric	Mathematical Definition	Operational Significance
MAE (Mean Absolute Error)	$MAE = (1 / N) \sum \lvert y_pred_i - y_true_i \rvert$	Average number of people by which the model's estimate deviates from the true manual head count per frame.
RMSE (Root Mean Squared Error)	$RMSE = \sqrt{ [(1 / N) \sum (y_pred_i - y_true_i)^2] }$	Penalizes large outlier estimation errors more heavily than small fluctuations.
Mean Signed Error (Bias Indicator)	$Bias = (1 / N) \sum (y_pred_i - y_true_i)$	Determines whether the detector has a systemic tendency to under-count (Bias < 0) due to occlusion or over-count (Bias > 0) due to false positive detections.
WAPE (Weighted Absolute % Error)	$WAPE = \sum \lvert y_pred_i - y_true_i \rvert / \sum y_true_i$	Total absolute error normalized by the total crowd volume. Avoids division-by-zero instability common in standard MAPE when frame counts are small.
Counting Accuracy %	$Accuracy = \max(0, \quad 100 \times (1 - WAPE))$	Executive headline metric representing total fidelity of the crowd estimation within the monitored zone.

6. Project Presentation & Viva Defense Guide

> **Key Talking Points for Presentation Slides**
>
> 1. **Practical Focus:** Highlight that MetroGuard solves the immediate, mission-critical problem of station congestion and surge management using existing CCTV infrastructure.
> 2. **Algorithmic Integrity:** Emphasize the footpoint ground-contact filter, which prevents false alarms caused by people walking outside the platform area.
> 3. **Tracking Reliability:** Explain how ByteTrack preserves commuter identity through dense occlusions, preventing double-counting or erratic metric jumping.
> 4. **Scientific Grounding:** Showcase the `evaluate_counting.py` tool to prove that accuracy claims are grounded in reproducible empirical testing, not speculative marketing.

Anticipated Technical Questions & Model Answers

Q1: Why use YOLOv11s instead of YOLOv11x or a heavier Vision Transformer?
Answer: Real-time transit safety requires low-latency inference (typically ≥ 20 FPS per camera). YOLOv11s achieves optimal Pareto-efficiency: high recall for human commuters while maintaining low computational overhead, enabling multiple video streams to run concurrently on standard GPU hardware or edge accelerators. Furthermore, by increasing

inference resolution to `imgsz=1280`, small and distant commuters are reliably detected without incurring the latency penalty of larger model backbones.

Q2: Why does the system report "occupancy per megapixel" rather than "people per square meter"?
Answer: Calculating physical density (people/m^2) requires calculating a homography matrix from camera calibration and measuring real-world floor landmarks. Reporting physical density without camera calibration produces mathematically invalid figures. MetroGuard AI reports true image-space occupancy in Phase-I and provides configurable operational thresholds (T_med, T_high) that operators calibrate per installed camera.

Q3: How does ByteTrack handle commuter occlusion on crowded platforms?
Answer: Traditional trackers (like SORT or DeepSORT) discard detections whose confidence score falls below a fixed threshold (e.g., 0.5). When commuters overlap, their confidence often drops to 0.2–0.4 due to partial visibility, causing classic trackers to drop the track and create a new ID upon reappearance. ByteTrack performs two-stage association: it matches high-confidence boxes first, and then matches remaining unmatched tracks against the low-confidence detections, preserving track continuity through heavy crowd occlusions.

7. System Requirements & Operational Verification

7.1 Environment Setup

```
# Create and activate Python 3.12 virtual environment
python -m venv .venv
.\\.venv\\Scripts\\Activate.ps1

# Install core runtime dependencies
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

7.2 Quick Run Commands

Operational Mode	Execution Command
Interactive Dashboard	`.\\.venv\\Scripts\\python.exe -m streamlit run dashboard.py`
CLI Video Processing	`.\\.venv\\Scripts\\python.exe run_video.py "path\\to\\crowd_clip.mp4" --max-frames 150`
Evaluation (Sampling)	`.\\.venv\\Scripts\\python.exe evaluate_counting.py prepare "path\\to\\video.mp4" --samples 20`
Evaluation (Scoring)	`.\\.venv\\Scripts\\python.exe evaluate_counting.py score evaluation/count_labels.csv`

7.3 Phase-II Evolution Roadmap

- **Four-Point Planar Homography:** Allow operators to click four ground points on the dashboard to map 2D camera perspective coordinates directly to 3D station floor blueprints, enabling physical people/m^2 density calculations.
- **Multi-Zone Polygonal ROIs:** Support arbitrary convex polygons (e.g., platform edge hazard zones, escalator entry queues) with differentiated alert thresholds.
- **TensorRT Acceleration:** Export the YOLOv11s weights to FP16 TensorRT engine formats for high-density multi-camera edge server deployments.